

北京理工大学

本科生毕业设计（论文）

LeRobot 平台感知—推理—执行流程的 系统级性能剖析研究

System-Level Performance Profiling of
Perception-Inference-Execution Pipeline
in the LeRobot Framework

学 院： 计算机学院

专 业： 软件工程

班 级： 08012204

学 生 姓 名： 俞乐楠

学 号： 1120221303

指 导 教 师： 王勇

2026 年 4 月 20 日

原创性声明

本人郑重声明：所呈交的毕业设计（论文），是本人在指导老师的指导下独立进行研究所取得的成果。除文中已经注明引用的内容外，本文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。

特此申明。

本人签名：_____ 日期：_____ 年 ____ 月 ____ 日

关于使用授权的声明

本人完全了解北京理工大学有关保管、使用毕业设计（论文）的规定，其中包括：学校有权保管、并向有关部门送交本毕业设计（论文）的原件与复印件；学校可以采用影印、缩印或其它复制手段复制并保存本毕业设计（论文）；学校可允许本毕业设计（论文）被查阅或借阅；学校可以学术交流为目的，复制赠送和交换本毕业设计（论文）；学校可以公布本毕业设计（论文）的全部或部分内容。

本人签名：_____ 日期：_____ 年 ____ 月 ____ 日

指导教师签名：_____ 日期：_____ 年 ____ 月 ____ 日

LeRobot 平台感知—推理—执行流程的 系统级性能剖析研究

摘 要

[TODO: 正式摘要待实验完成后撰写, 约 300-500 字。]

本文以树莓派 5 (ARM Cortex-A76, 无 GPU) 为目标平台, 对 LeRobot 框架下 ACT 机器人学习策略的感知—推理—执行三阶段时序特征进行系统级剖析。通过在 Python 应用层、PyTorch 运行时层、Linux 内核层及硬件 I/O 层的多工具联合测量 (Python logging / torch.profiler / ftrace / strace), 端到端量化了各阶段的时间开销, 定位了主要性能瓶颈, 并从算法、运行时、系统架构三个维度梳理了可行的优化路径。

实验结果表明, ACT 单次推理耗时 2000–4000 ms, 占总循环时间的 99% 以上, 是系统主瓶颈; 观测和执行阶段开销极低 (各约 5 ms), 系统调用本身的开销可忽略不计, 排除了 I/O 路径为主要瓶颈的假设。在 chunk_size=100 配置下, 系统实际帧率约 18–19 fps, 距 30 fps 目标仍有明显差距, 瓶颈根源在于 ARM CPU 矩阵乘法 (GEMM) 计算能力不足, 而非感知复杂度或任务类型。

关键词: LeRobot; ACT; 性能剖析; ARM 推理; chunk_size; 感知-推理-执行

System-Level Performance Profiling of Perception-Inference-Execution Pipeline in the LeRobot Framework

Abstract

[TODO: English abstract to be finalized after experiments complete, approximately 300 words.]

This paper presents a system-level performance analysis of the ACT (Action Chunking with Transformers) policy running on a Raspberry Pi 5 (ARM Cortex-A76, no GPU) within the LeRobot framework. By combining multi-layer profiling tools across the Python application layer, PyTorch runtime layer, Linux kernel layer, and hardware I/O layer (Python logging, torch.profiler, ftrace, strace), we quantify the end-to-end time distribution across perception, inference, and execution stages, identify the primary bottleneck, and systematically evaluate feasible optimization paths.

Results show that a single ACT inference takes 2000–4000 ms, accounting for over 99% of the total loop time, making it the dominant bottleneck. Perception and execution stages contribute only ≈ 5 ms each; system call overhead is negligible, ruling out I/O as a bottleneck. At `chunk_size=100`, the system achieves ≈ 18 – 19 fps against a 30 fps target. The root cause is insufficient ARM CPU matrix-multiplication (GEMM) throughput, independent of task visual complexity or object count.

Key Words: LeRobot; ACT; performance profiling; ARM inference; `chunk_size`; perception-inference-execution

目录

第一章 引言	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 机器人通信框架	2
1.2.2 开源机器人硬件平台	3
1.2.3 机器人策略模型	3
1.2.4 策略系统架构：端到端与显式分解式	5
1.2.5 嵌入式边缘 AI 推理	6
1.2.6 系统级 Profiling 技术	6
1.3 研究目标	7
1.4 主要贡献	7
第二章 实验环境构建	7
2.1 感知—推理—执行三阶段划分	7
2.2 LeRobot 软件栈架构	8
2.3 三类典型工作负载特征	9
2.4 跨层次性能分析模型	10
2.5 工具链选型	11
2.6 实验硬件与软件配置	15
第三章 性能分析	16
3.1 感知阶段性能剖析	16
3.1.1 摄像头异步读取机制	17
3.1.2 是否进入内核	17
3.1.3 OpenCV 到内核的边界	18
3.1.4 时间剖析	19
3.2 推理阶段性能剖析	19
3.2.1 <code>select_action</code> 与动作队列	20
3.2.2 一次完整 ACT 推理的数据流	21
3.2.3 时间剖析： <code>chunk_size</code> 参数扫描	21
3.2.4 单次 ACT 推理耗时分解	23

3.2.5	PyTorch 线程行为与推理优化研究现状	24
3.3	执行阶段性能剖析	24
3.3.1	舵机感知读取（sync_read 路径）	24
3.3.2	舵机指令下发（sync_write 路径）	25
3.3.3	半双工总线协议特性与瓶颈	26
第四章	优化	26
4.1	性能瓶颈归因	26
4.2	跨阶段综合分析	26
4.2.1	时间预算表（单帧，33.3 ms 目标）	26
4.2.2	系统调用分布（40 s episode）	27
4.3	异步化可行性调研	27
4.3.1	摄像头异步读	27
4.3.2	推理异步	27
4.3.3	舵机异步读	28
4.3.4	舵机异步写	28
4.4	优化方向矩阵	29
4.5	软件架构改进方向	29
第五章	总结与展望	29
5.1	主要工作总结	29
5.2	研究局限性	30
5.3	未来工作	30
	参考文献	31
	附录	32
	致谢	33

第一章 引言

1.1 研究背景与意义

近年来，协作机器人在工业制造、医疗服务、仓储物流等领域加速落地，对自主操作能力的需求持续增长。以 RT-2^[1]、OpenVLA^[2] 为代表的视觉—语言—动作（Vision-Language-Action, VLA）大模型，将视觉感知与动作决策统一到端到端网络中，显著提升了机器人在开放场景下的泛化能力；然而，此类模型的参数量和计算量远超传统控制方法，如何在资源受限的嵌入式平台上实现实时推理，成为制约其部署的关键瓶颈。

在开源机器人学习框架方面，HuggingFace 推出的 LeRobot^[3] 集成了 ACT（Action Chunking with Transformers）^[4]、Diffusion Policy^[5] 等模仿学习策略，并提供从数据采集、模型训练到在线推理的完整工具链；同时配套推出 SO-101 开源六自由度机械臂，实现了软件框架与低成本硬件平台的一体化开源。Physical Intelligence 提出的 π_0 ^[6] 以流匹配为动作生成核心，在双臂协作任务上表现出色；斯坦福 Mobile ALOHA^[7] 以低成本仿生平台搭配联合学习，实现了全身遥操作控制。LeRobot 的在线推理循环采用“感知—推理—执行”三阶段紧耦合架构：每帧依次完成传感器观测采集（`get_observation()`）、策略网络推理（`select_action()`）和舵机指令下发（`send_action()`），三个阶段在同一线程内串行执行，任一阶段的性能瓶颈均会直接影响整体帧率。

在实际部署中，本文选用树莓派 5（ARM Cortex-A76，4 核，4 GB LPDDR4X）作为边缘推理平台，搭配 SO101 六自由度机械臂和 USB 摄像头，在无 GPU 的条件下运行 ACT 策略网络（开源硬件平台对比详见 §1.2.2）。初步测试表明，单次 ACT 推理耗时约 1500-3000 ms，而传感器采集和指令下发各仅约 1-15 ms，推理阶段占循环总时间的 99% 以上。在 `chunk_size=100`（即每次推理输出 100 帧动作）的配置下，系统实际帧率仅约 18-19 fps，距离 30 fps 的实时目标仍有明显差距。

上述性能瓶颈涉及多个系统层次：PyTorch 推理引擎的算子调度、OpenMP 线程池的并行效率、Linux 内核的系统调用路径以及 USB/串口的硬件 I/O 延迟。然而，现有的机器人系统性能分析工作多聚焦于单一层次（如模型压缩、内核调度或通信延迟），缺乏从 AI 框架层到内核层的跨层次联合剖析，难以回答“瓶颈究竟在哪一层、由什么原因导致、各优化方向的收益几何”等关键问题。

因此，本文的研究意义在于：（1）理论层面——建立覆盖 AI 推理框架层、Python

运行时层、OS 内核层和硬件 I/O 层的四层性能分析模型，提出面向“感知—推理—执行”紧耦合场景的多工具联合剖析方法；(2) 实践层面——端到端量化 LeRobot 在嵌入式 ARM 平台上的各阶段时间开销，定位主要性能瓶颈，系统评估 INT8 量化、异步推理流水线等优化方向的可行性与预期收益，为同类嵌入式机器人系统的性能优化与软件架构设计提供实验依据和工程参考。

1.2 国内外研究现状

本节从机器人通信框架、开源机器人硬件、机器人策略模型、嵌入式边缘 AI 推理、系统级 Profiling 技术五个维度梳理与本文相关的研究现状。

1.2.1 机器人通信框架

机器人系统通常由传感器采集、策略决策、运动控制、人机交互等多个功能模块组成，这些模块运行在不同的进程甚至不同的物理机器上，需要一种**标准化的通信机制**来协调数据流转。ROS¹ 提供统一的发布-订阅消息总线、服务调用接口和参数配置服务，使各功能模块可以作为独立节点开发、部署和升级，底层仍依赖 Linux 内核进行进程调度和设备管理。在多传感器融合、多机协作、导航 + 操作叠加等场景下，这种通信抽象能显著降低系统集成复杂度，也是 ROS 系列成为学术界和工业界事实标准的原因。

然而，引入 ROS 这类中间件层也带来不可忽视的代价：(1) **通信延迟**：消息需要经过序列化、DDS 传输、反序列化三个阶段，单次跨节点通信通常引入 1–5 ms 延迟，在帧间隔仅 33.3 ms (30 fps) 的实时控制场景中不可忽略；(2) **资源开销**：DDS 守护进程和 ROS2 节点管理器占用额外的 CPU 和内存，在树莓派等资源受限平台上会挤压推理和感知的算力预算；(3) **部署复杂度**：需要构建 ROS2 工作空间、配置 DDS QoS 策略、维护节点间的 topic 命名约定，相比直接在 Linux 上编写 Python 脚本，开发门槛和调试成本显著提高。

因此，**是否引入中间件通信框架取决于具体场景**：多机协作、传感器异构、需要 rviz 可视化和导航栈的场景适合 ROS2；单机单臂、帧率敏感、追求极简部署的场景则更适合直接在 Linux 用户态运行。本文的 LeRobot 平台属于后者——单进程 Python 脚本串行执行感知—推理—执行三阶段，无分布式需求，且第三章的剖析将表明通信开销远低于推理开销，引入 ROS2 不会带来性能收益。但了解 ROS2 的通

¹ROS 全称为 Robot Operating System，但并非操作系统，而是一套运行于 Linux 用户态之上的中间件通信框架。

信模型有助于理解 LeRobot 的架构选择及其局限性。

ROS2: 机器人操作系统 (Robot Operating System 2) 是 ROS 的下一代版本, 以 DDS (Data Distribution Service) 作为底层通信中间件, 支持节点间发布-订阅与服务调用。ROS2 引入了节点生命周期管理、QoS 策略 (可靠性/持久性/实时性配置) 等企业级特性, 支持跨机器跨平台分布式部署, 并可通过 `rclcpp` / `rclpy` 接入 C++ 或 Python 节点。其通信延迟在局域网内通常为毫秒级, 但节点调度仍依赖 Linux CFS 调度器, 无硬实时保证。ROS2 主要运行于 Ubuntu (官方支持 22.04/24.04), 需配合 Linux 内核使用, 不支持 Windows/macOS 原生部署。

LCM (Lightweight Communications and Marshalling): 由 MIT 开发的高性能消息传递库, 设计目标是为实时系统提供低延迟、零拷贝的消息序列化与传输能力。LCM 使用 UDP 单播/广播而非 TCP, 支持用户态消息队列 (类型声明 + 字节流), 省去了 IDL 编译步骤, 延迟可低至数微秒。但 LCM 仅提供消息传递层, 不含控制回路、参数服务器或 RViz 等生态工具, 生态远不如 ROS2。

[来源: *LeRobot* 软件栈全链路分析.md]

1.2.2 开源机器人硬件平台

SO-101: HuggingFace LeRobot 配套的六自由度机械臂, 结构件可 3D 打印, 成本低廉, 与 LeRobot 框架深度绑定, 是本文实验平台的直接来源。

OpenManipulator^[8]: ROBOTIS 推出的开源机械臂, 提供配套的 ROS2 集成包, 硬件图纸与代码全开源, 适合教学和原型开发, 支持 MoveIt! 运动规划。

TurtleBot4^[9]: 开源移动机器人的事实标准, 最新版本与 ROS2 生态深度集成, 配备 OAK-D 深度相机, 适用于室内导航与移动操作任务。

Berkeley Humanoid Lite^[10]: UC Berkeley HybridRobotics 实验室开发的 3D 打印小型人形机器人, 为开源人形机器人平台提供了新思路。

1.2.3 机器人策略模型

从动作生成或策略学习机制看, 机器人策略模型并非只有 VLA 和模仿学习两类, 还包括传统规划控制、强化学习等路线。这些方法关注的是动作如何产生、策略如何获得; 至于系统是否采用端到端或显式分解式架构, 则属于另一条系统架构维度, 将在下一小节单独讨论。

传统规划与控制方法: 通常先建立机器人运动学、动力学或环境几何模型, 再通过路径规划、轨迹优化和反馈控制生成动作。典型方法包括 PID 控制、模型预测控

制 (Model Predictive Control, MPC)、采样式路径规划和 MoveIt!/OMPL 等运动规划框架。这类方法可解释性强、实时性好、安全约束清晰, 适合结构化工业环境; 但在高维视觉输入、复杂接触和开放场景泛化方面通常需要额外的感知与任务规划模块支撑。

强化学习策略 (Reinforcement Learning, RL): 通过奖励函数引导智能体与环境交互, 学习状态到动作的映射, 不一定依赖专家演示数据。代表方法包括 PPO、SAC、TD-MPC 等。RL 适合处理长时序决策、非线性控制和需要主动探索的任务, 但真实机器人采样成本高、训练过程不稳定, 往往需要仿真环境、sim-to-real 迁移或安全约束机制。因此, 在低成本真实机械臂上直接使用 RL 训练策略的工程门槛较高。

模仿学习策略 (Imitation Learning, IL): 通过专家演示数据训练策略网络, 将人类遥操作或已有控制器生成的轨迹作为监督信号。本文采用的 ACT 即属于**端到端模仿学习策略**: 模型以相机图像和机械臂关节状态为输入, 直接预测未来一段动作序列, 不在线推理阶段显式调用目标检测、运动规划或传统控制模块。代表工作包括:

- **ACT** (Action Chunking with Transformers): CVAE 编解码器 + Transformer 动作预测, 支持动作分块 (chunk_size) 和 time ensemble, 适合桌面精细操作任务;
- **Diffusion Policy**: 将动作生成建模为扩散过程, 适合多模态动作分布。

视觉-语言-动作模型 (Vision-Language-Action, VLA): 在端到端动作预测的基础上进一步引入语言理解能力, 通常利用大规模视觉-语言模型或机器人基础模型进行预训练, 再将视觉、语言和动作统一到同一模型中。VLA 的优势在于开放词汇指令理解、多任务泛化和跨场景迁移能力; 代价是模型参数量和计算量显著高于普通模仿学习策略。代表工作包括:

- **RT-2**: Google DeepMind 提出, 将视觉-语言模型 (VLM) 输出直接映射为机器人动作 token;
- **OpenVLA**: 开源 VLA 模型, 支持多任务泛化, 参数量 7B, 可在消费级 GPU 部署;
- **π_0** : Physical Intelligence 提出, 以流匹配 (Flow Matching) 为核心动作生成机制, 在双臂协作任务上表现优异。

对比：VLA 泛化能力强但计算量巨大（7B+ 参数），难以在嵌入式 ARM 平台实时运行；RL 具备交互式优化能力，但真实机器人训练成本高且稳定性要求严格；传统规划控制实时性好，但对开放视觉场景和复杂接触任务的适应性有限。相比之下，ACT/Diffusion Policy 等端到端模仿学习策略参数量较小（ResNet18 + Transformer, 约 50M 参数），可在树莓派 5 上运行，但推理延迟仍是主要瓶颈（2000–4000 ms/次）。因此，本文聚焦 LeRobot 框架下 ACT 策略在 ARM CPU 平台上的在线推理性能剖析。

1.2.4 策略系统架构：端到端与显式分解式

与按学习机制划分不同，系统架构描述的是机器人系统内部如何组织感知、决策、规划和控制模块。从这一维度看，可以将机器人策略系统概括为**端到端架构**与**显式分解式架构**两类。前者尽量用一个统一模型直接从观测输入映射到动作输出；后者显式保留中间状态、任务分解、轨迹规划或底层控制接口。模块化、层级式和混合式并不是三个彼此互斥的类别，而是显式分解式架构在功能边界、决策层级和模块组合方式上的不同侧面。

端到端架构：模型直接从观测输入（如图像、关节状态、语言指令）预测机器人动作，中间不显式拆分为目标检测、状态估计、轨迹规划和控制器等人工设计模块。其优势是系统链路短、手工建模较少，便于从大规模数据中学习复杂的视觉—动作映射；不足是可解释性和安全约束较弱，对训练数据分布和推理算力依赖较强。ACT、Diffusion Policy 和许多 VLA 模型都可以采用端到端架构，强化学习策略也可以用端到端神经网络实现。

显式分解式架构：系统显式拆分为感知、状态估计、任务规划、轨迹生成和底层控制等环节，各环节之间通过中间表示传递信息。这类架构可解释性强、便于调试和加入安全约束，常见于工业机器人、导航系统和基于 MoveIt! 的操作系统；但模块边界和中间表示需要人工设计，前级感知误差也可能沿链路逐级传播。在具体系统中，它通常体现为几个并存的组织特征：**模块化**强调按功能边界拆分系统，例如感知、规划、控制分别由不同模块实现；**层级式**强调按决策抽象层次或时间尺度拆分系统，高层负责语言理解、任务分解和目标选择，低层负责连续动作生成与控制执行；**混合式**强调在同一系统中组合学习模型与显式规划控制模块，例如用 VLA/LLM 进行高层任务规划，用 ACT 或 Diffusion Policy 执行低层动作，再用传统控制器处理限位保护、速度约束和急停逻辑。

因此，模块化、层级式和混合式系统整体上通常与纯端到端系统相对，但并不意

意味着它们不能使用端到端模型。相反，复杂机器人系统常常在某个局部子模块中使用端到端策略：例如高层模块决定“抓取杯子并放到托盘上”的子任务顺序，低层 ACT 策略负责每个子任务中的连续关节动作生成。

本文实验系统采用的是较简洁的端到端模仿学习架构：LeRobot 在线循环中，ACT 策略根据相机图像和关节状态直接输出动作序列，系统不额外引入高层任务规划器、显式轨迹优化器或 ROS2/MoveIt! 等模块化控制链路。这使得性能瓶颈更集中地体现为 ARM CPU 上的策略网络推理开销。

[来源：ACT 模型原理.md, ACT 推理.md]

1.2.5 嵌入式边缘 AI 推理

在资源受限的嵌入式平台上部署深度学习模型，需要针对硬件特性进行优化：

TFLite: Google 推出的移动端推理框架，支持 FP16/INT8 量化、ARM NEON SIMD 加速，广泛用于边缘设备推理。

ONNX Runtime: 微软推出的跨平台推理引擎，支持多种执行提供者（CPU/CUDA / TensorRT），在 ARM 上通过 OpenMP 和 NEON 加速，延迟优于 TFLite 约 1.5-2 $\times$ 。

PyTorch Mobile: PyTorch 的移动端部署方案，支持 TorchScript 导出和 CPU 优化，但相比 ONNX Runtime 在 ARM 上的优化程度较低。

树莓派 5 (Cortex-A76) 计算能力: 4 核 4 GB LPDDR4X，双发射 SIMD 单元支持 NEON，矩阵乘法 (GEMM) 理论峰值约 50 GFLOPS (FP32)，实际推理 ACT 模型约 30-50 GFLOPS 利用率，实测推理耗时 2000-4000 ms/次。作为对比，RT-2 (7B 参数) 在树莓派 5 上几乎无法运行，VLA 模型需在 GPU 平台上部署。

1.2.6 系统级 Profiling 技术

机器人系统的性能分析需要跨层次工具链：

- **strace**: 通过 ptrace 拦截系统调用，开销极大 (9 $\times$ 时间放大)，仅用于定性探索；
- **ftrace**: 内核态 tracepoint，开销极低 (纳秒级)，适合生产环境测量；
- **perf**: 硬件性能计数器 (PMC)，可测量 IPC、Cache-miss 等微架构指标；
- **torch.profiler**: PyTorch 算子级 profiling，分解前向/反向传播各算子耗时。

已有机器人系统性能分析工作多聚焦于单一层次（模型压缩、内核调度或通信延迟），缺乏从 AI 框架层到内核层的跨层次联合剖析。

[来源：内核数据搜集分析.md, 方法论.md]

1.3 研究目标

1. 建立跨层次（AI 框架层 / 运行时层 / OS 内核层 / 硬件 I/O 层）性能分析模型；
2. 端到端量化感知、推理、执行各阶段性能开销；
3. 定位主要瓶颈，为系统优化与软件架构改进提供实验依据。

1.4 主要贡献

- 提出覆盖 AI 框架到内核的四层性能分析模型；
- 通过 ftrace/torch.profiler/perf 多工具联合剖析，量化各阶段时间开销；
- 揭示 strace 对系统调用测量带来的 9× 时间放大问题，验证 ftrace 的必要性；
- 基于量化结果，系统梳理优化路径及其成本收益。

第二章 实验环境构建

LeRobot 是 HuggingFace 推出的开源机器人学习平台，提供从数据采集、模型训练到在线部署的完整工具链，支持模仿学习（ACT、Diffusion Policy）和强化学习（SAC、TDMPG）等多种策略。LeRobot 聚焦于低成本硬件场景，配套 SO-101 六自由度机械臂，只需一块树莓派和一台 3D 打印机即可搭建完整的实验环境，本文选用 LeRobot 作为实验平台。

2.1 感知—推理—执行三阶段划分

LeRobot 的在线推理过程是一个持续运行的帧循环，每一帧依次经历**观测采集**（`get_observation()`）、**策略推理**（`select_action()`）、**指令下发**（`send_action()`）和**等待**四个阶段，周而复始直到 episode 结束。其中观测阶段同时采集两类传感器数据：USB 摄像头的图像帧和机械臂 6 个关节的角度位置；摄像头图像通过 OpenCV 后台线程异步预取，主线程调用 `async_read()` 时直接取回已解码的最新帧，关节角度则

通过 `scservo_sdk` 以 `sync_read` 指令从舵机总线同步读取。该循环在单个 Python 进程内串行执行，不依赖外部消息中间件，这一架构选择作为第三章剖析的基本假设。图 1 展示了单帧循环的执行流程。

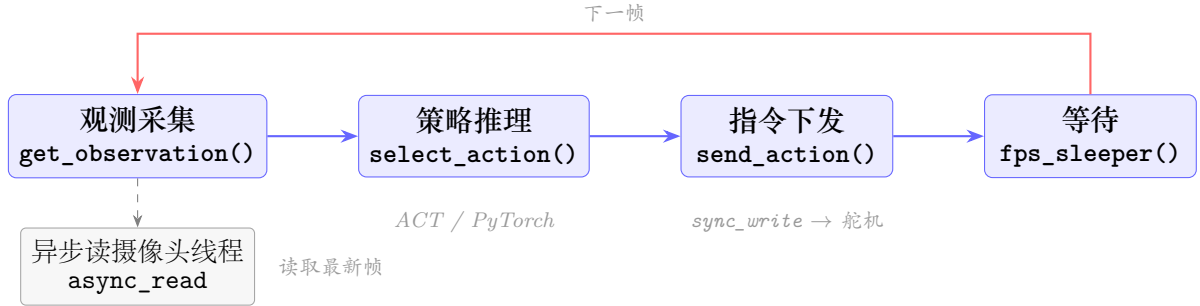


图 1: LeRobot 单帧推理循环流程：观测 → 推理 → 执行 → 等待，周而复始

从代码实现看，循环中的主要阶段边界分别对应 `get_observation()`、`select_action()` 和 `send_action()`。若以 30 fps 作为在线控制目标，则单帧可用时间约为 33.3 ms。在当前 `chunk_size=100` 的配置下，实测帧率约为 18–19 fps，其中策略推理约占总时间的 37%，等待阶段约占 42%。

2.2 LeRobot 软件栈架构

图 2 展示了上述推理循环运行时所依赖的软件栈分层结构。顶层为应用逻辑层，包含执行、观测、推理三个功能模块；中间层为支撑各模块的第三方库与驱动——执行和观测中的舵机操作通过 `scservo_sdk` / `pyserial` 驱动，观测中的图像采集通过 OpenCV 完成（含后台异步读线程），推理阶段依赖 PyTorch 运行策略网络；底层依次为 glibc 系统库和 Linux 内核。

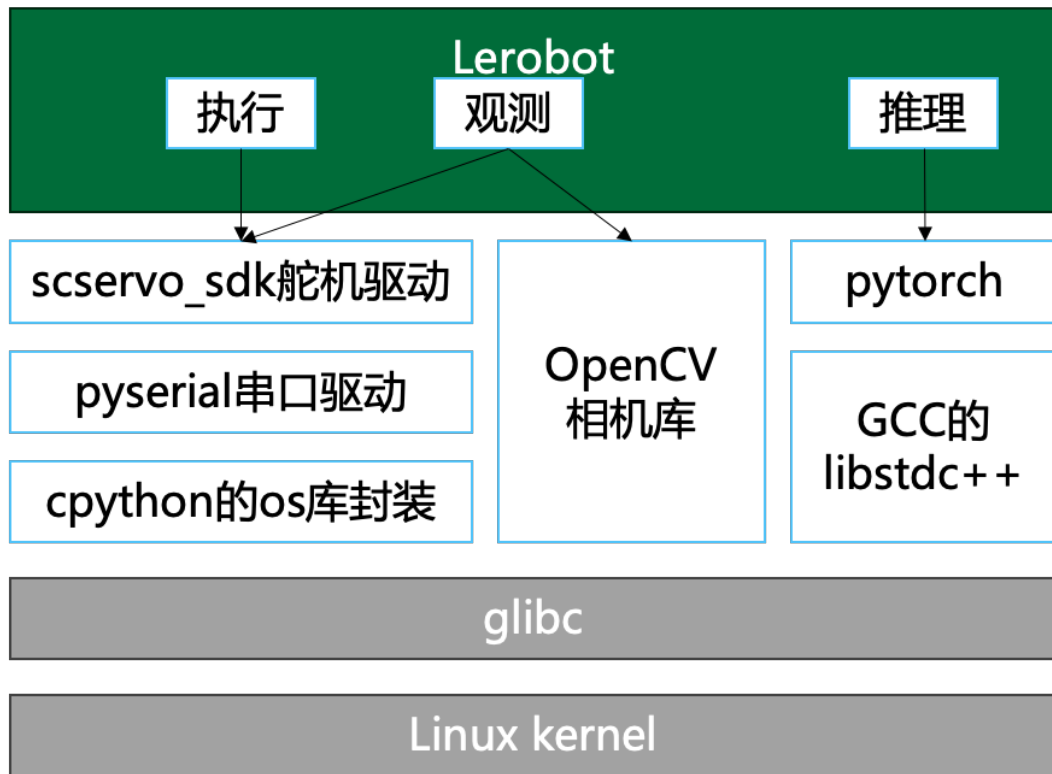


图 2: LeRobot 软件栈四层架构：应用逻辑层 → 库与驱动层 → glibc → Linux 内核

[来源: *LeRobot 软件栈全链路分析.md*]

2.3 三类典型工作负载特征

三类任务：抓取 (Grasp / pick)、推倒 (Push)、分类 (Sort / classification)。统一硬件（树莓派 5）、统一模型架构（ACT，cs=100）、统一目标帧率（30 fps），每类任务采集 $n = 20$ 个 episode，episode 时长统一对齐至约 30 s 以排除时长差异干扰。本节使用实验 02 的统计结果作为三类工作负载的基准画像，只比较各阶段时间占比，而不把 episode 绝对时长差异混入分析。

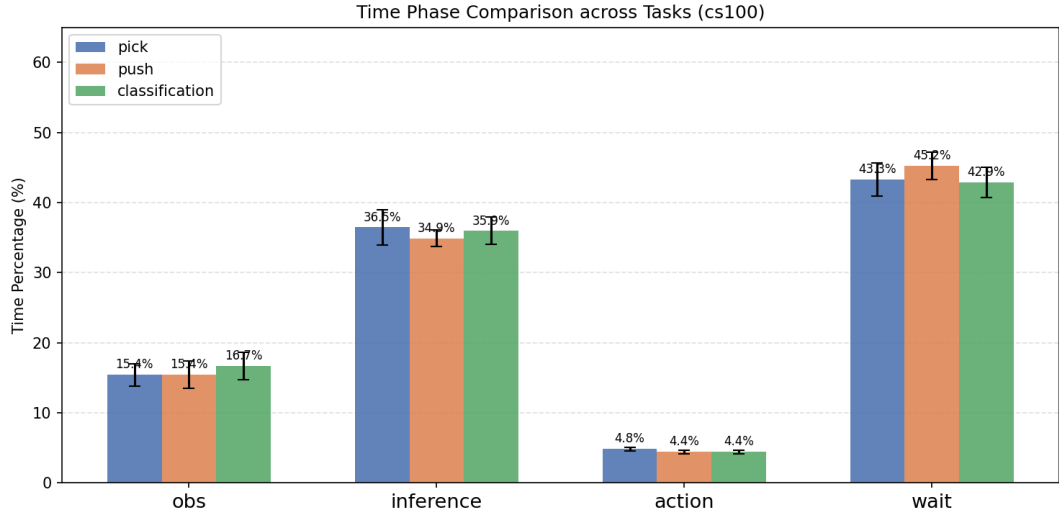


图 3: 三类任务在 `chunk_size=100` 下的阶段时间占比对比

图 3 的主要现象有三点。第一，三类任务的推理占比都集中在 35%–36% 左右，说明在模型结构、`chunk_size` 和输入配置相同的情况下，ACT 单次推理开销主要由模型本身决定，而不是由抓取、推倒、分类这些任务语义决定。第二，推倒任务的 `wait_pct` 最高，达到 45.2%，比抓取和分类高约 2 个百分点；这更像是动作行程和执行节奏造成的等待增加，而不是推理侧变慢。第三，`obs_pct` 和 `action_pct` 在三类任务之间变化很小，说明相机读取、舵机状态读取以及舵机写入在当前实验设置下更接近固定开销。

因此，后续性能分析不把三类任务视为三种完全不同的软件路径，而是把它们看作同一 LeRobot 推理循环在不同动作负载下的表现。抓取和分类更接近基准工作负载；推倒任务由于等待占比最高，更适合用来观察执行节奏和 `chunk_size` 对整体帧率的影响。

[来源：实验/02_ 三类任务工作负载对比/`cs100_task_comparison.md`]

2.4 跨层次性能分析模型

层级	可观测指标	工具
AI 推理框架层	算子耗时、张量形状、FLOP	torch.profiler
Python 运行时层	GIL 争用、帧循环时间	Python logging
OS 内核层	系统调用次数/耗时、线程调度	ftrace, strace
硬件 I/O 层	传输字节数、等待时延	perf, iocli trace

2.5 工具链选型

strace 与 ftrace 机制对比：

两者作用层次不同，开销来源也不同：

- **strace**：通过 `ptrace` 系统调用在用户态拦截目标进程的每一次系统调用入口与出口。每次拦截都需要一次进程上下文切换（目标进程 → `strace` 进程），其开销与系统调用频率正相关，频繁调用时对主循环干扰极大，不能作为定量时序数据来源（具体扰动量级见后文实测）。
- **ftrace**：通过 Linux 内核 `tracepoint` 在内核态直接写入 `ring buffer`，插桩本身开销极低（纳秒级，无进程切换）。但将 `ring buffer` 内容导出至用户态文件（`cat /sys/kernel/debug/tracing/trace > log` 或后台进程持续读取 `trace_pipe`）是一次批量/持续文件 I/O，当 `event` 密集时，导出过程本身会消耗额外 CPU 和内存带宽，且实时导出期间 `ring buffer` 也可能覆盖旧数据（丢失事件）。

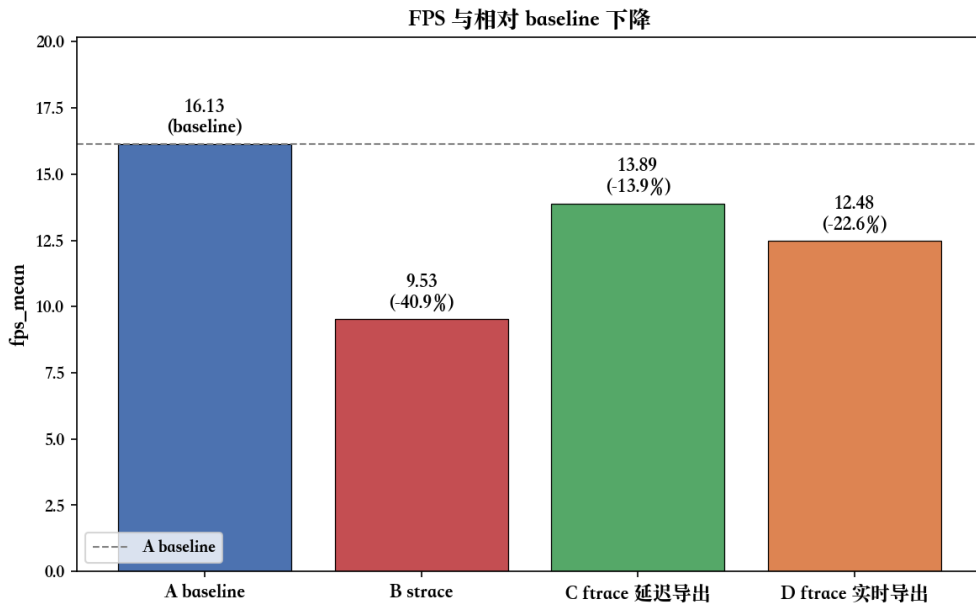
测量工具开销实测

为定量比较各工具对主循环的扰动，本文设计了一个测量工具校准实验。在固定 `chunk_size=100`、目标 `fps=30`、`episode` 时长约 30 s 的同一条 `record_loop` 上，对比四组方案：A 组 `baseline` 不开任何 `tracing`；B 组用 `strace -f -ttT` 包住 `record` 命令；C 组开启 `ftrace` 仅写内核 `ring buffer`，`episode` 结束后再导出 `trace`；D 组开启 `ftrace` 并同时启动后台进程实时读取 `trace_pipe` 写入文件。所有指标由 `record.py` 内部通过 `resource.getrusage()` 与 `sysfs` 自动采集，写入 `analysis/timing_stats.csv`。表 1 汇总了四组的核心指标，后文分别从 FPS、上下文切换次数和阶段占比漂移三个角度逐一分析。

表 1: 四组测量工具对 `record_loop` 的扰动（每组 $n = 1$ ，仅用于工具选型）

指标	A baseline	B strace	C ftrace 延迟	D ftrace 实时
<code>fps_mean</code>	16.13	9.53	13.89	12.48
相对 A 的 FPS 下降	—	-40.9%	-13.9%	-22.6%
<code>system_time_s</code>	1.40	2.76	1.37	1.03
<code>voluntary_cs</code>	23,947	750,662	30,377	36,336
<code>involuntary_cs</code>	78,436	541,334	84,481	52,907
<code>obs_pct</code> 漂移 (pp)	0	+34.8	-5.5	-3.0
<code>wait_pct</code> 漂移 (pp)	0	-31.6	-0.9	-0.8
trace 文件大小	—	157 MB	168 MB	823 MB

FPS 下降 图 4 给出了四组的实测 `fps_mean` 与相对 A 组 baseline 的下降幅度。B strace 组从 16.13 fps 跌至 9.53 fps，下降 40.9%，单这一项就足以排除 strace 作为正式时序数据来源的可能性。C ftrace 延迟导出组下降 13.9%，D ftrace 实时导出组下降 22.6%，两者都仍然超过设计文档定义的 $\leq 5\%$ 验收阈值；但需要注意每组只有 1 个 episode，A 组 baseline 自身存在抖动（如观测阶段单次冷启动时延变长会直接拉低绝对 FPS），正式实验需补足每组 5 个 episode 后再核对方差。即便如此，C 组优于 D 组、且两组都明显优于 B 组的相对秩序非常稳定，能够支撑后续的工具选型。

图 4: 四组测量工具下的 `fps_mean` 与相对 A 组 baseline 的下降百分比

上下文切换次数 图 5 用对数坐标对比了四组的自愿与非自愿上下文切换次数。B strace 组的 `voluntary_cs` 从 baseline 的约 2.4 万次飙升至约 75 万次（约 31 倍），`involuntary_cs` 也从 7.8 万次升至 54 万次（约 7 倍）。这种几何级放大是 `ptrace` 机制的直接体现：每一次系统调用都被强制中断到 `strace` 进程读取参数后再恢复，因此摄像头链路中密集出现的 `pselect6`、`read`、`ioctl` 都会在切换次数上留下印记。相比之下，C 组的切换次数仅比 baseline 高约 27%，D 组的额外切换主要来自后台 `cat trace_pipe` 进程，两者的量级都与 baseline 同档，说明 `ftrace` 内核 ring buffer 插桩本身对调度路径几乎无干扰。

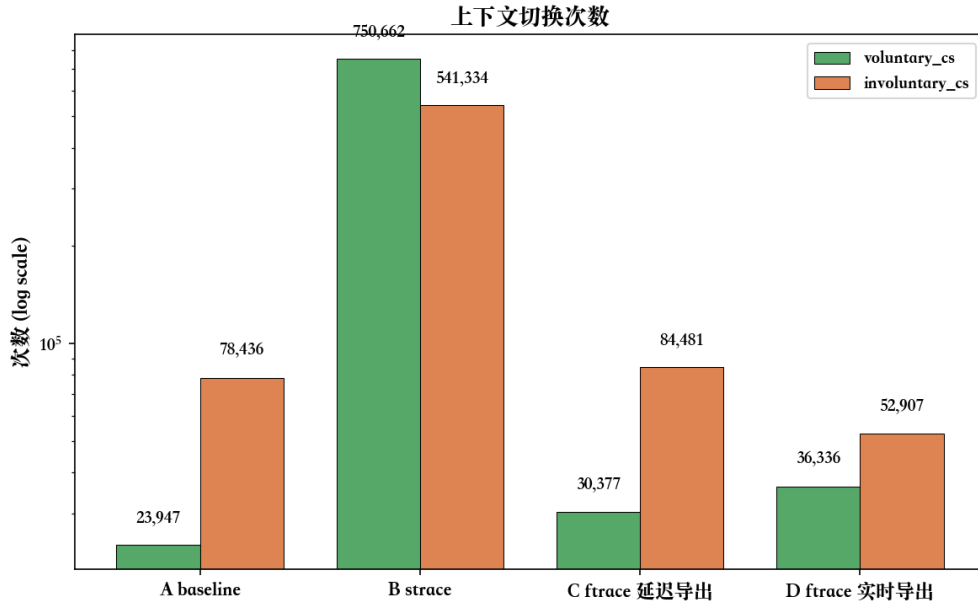


图 5: 四组的自愿与非自愿上下文切换次数（对数坐标）

阶段占比漂移 图 6 显示了四组各阶段占比相对 A 组 baseline 的偏移（pp = percentage point）。B strace 组的 `obs_pct` 漂移高达 +34.8 pp，`wait_pct` 反向漂移 -31.6 pp，意味着 `strace` 不仅整体拉慢主循环，还改变了阶段时间结构：摄像头链路中的系统调用被 `ptrace` 反复拦截，导致观测阶段被严重拉长，`wait` 阶段反而被挤压；这种结构性扭曲使得 `strace` 下采集的阶段占比已经无法代表真实负载。C 与 D 两组的所有阶段漂移都在 ± 8 pp 内，其中 `wait/action` 漂移均 ≤ 1 pp，`obs_pct` 与 `inference_pct` 在 ± 7 pp 之间小幅互相借调，没有出现 B 组那种数量级的结构变化；因此可以判断 `ftrace` 两种方案对阶段时间结构的影响可控。

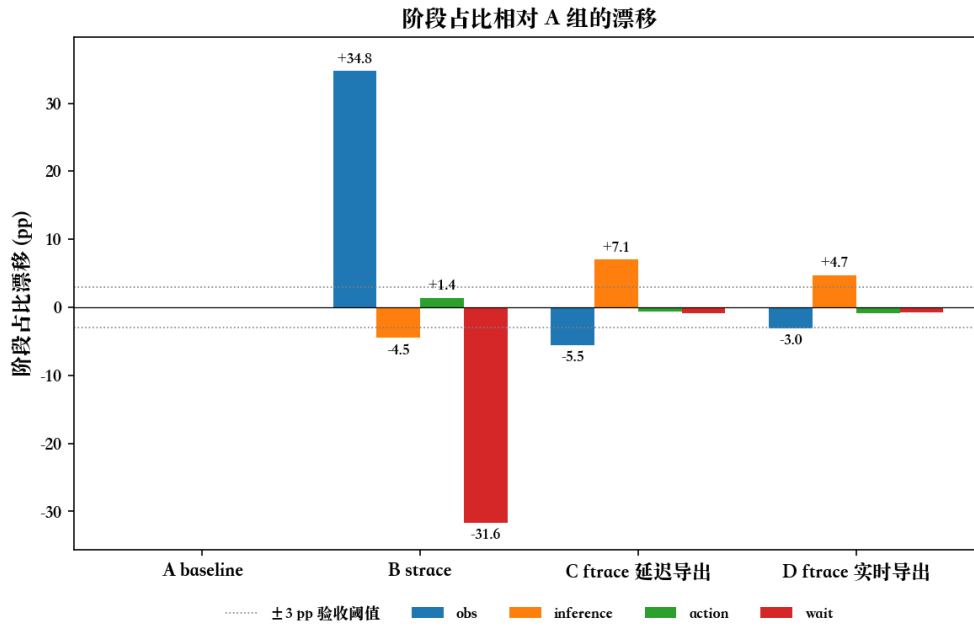


图 6: 四组的四阶段占比相对 A 组 baseline 的漂移 (pp), 灰色虚线为设计阶段提出的 ± 3 pp 验收阈值

本文工具链选型 正式实验的内核态时序数据采用 **ftrace 延迟导出** 方案: 其上下文切换次数与阶段时间结构均与 baseline 同档, 是单 episode 扰动最小的方案。仅在 episode 时长延长导致 ring buffer (本文配置每 CPU 32 MB, 4 核共 128 MB) 接近溢出时, 才改用 ftrace 实时导出作为后备; 其代价是 trace 数据量约为延迟导出的 4.9 倍 (823 MB vs 168 MB), 但量级仍可承受。strace 不参与最终定量分析, 仅用于第三章早期对系统调用类型分布的定性探索。

内核源码级插桩 除上述标准 syscall tracepoint 外, 本文还在内核源码中通过 `trace_printk` 插入了自定义计时桩, 将更精细的时间拆分信息直接写入 ftrace ring buffer, 与标准 tracepoint 事件混合输出。标准 tracepoint 只记录系统调用入口参数和出口返回值, 无法拆分调用内部的阻塞睡眠时间与 CPU 处理时间, 也无法将文件描述符解析为设备名。

本文在两处关键路径上实现了自定义插桩:

1. **pselect6** (`fs/select.c`, `do_select()` 函数): 在函数入口通过 `ktime_get()` 记录起始时间, 在 `poll_schedule_timeout()` 前后打点计算阻塞睡眠时间, 函数退出时输出总耗时、睡眠时间、上下文处理时间、返回值、就绪 fd 编号及通过 `d_path()` 解析的设备名。例如, trace 中可直接观察到 “`fd=8, file=video0`” 表

示手眼摄像头帧就绪，“fd=9, file=video2”表示固定摄像头帧就绪，每次等待约 30 ms，睡眠时间约占总耗时的 99.7%。

2. **futex(kernel/futex/syscalls.c, do_futex()) 函数**: 类似地在入口和 **futex_wait_multiple** 前后打点，输出操作码 (op)、返回值、总耗时、CPU 时间和睡眠时间。该拆分可以直接区分 futex 快速解锁 (sleep≈0) 与长时间等待 (sleep 接近 total)，从而判断 Python 线程间的锁竞争程度。实测数据显示 WAKE 操作 (op=1) 耗时在微秒级，而 WAIT 操作 (op=0) 的睡眠时间最高可达 1.8 s。

上述信息无法通过标准 tracepoint 或 kprobe 动态插桩获得：标准 tracepoint 不提供内部时间拆分；kprobe 回调中调用 **d_path()** 解析 fd 路径存在死锁风险，而 **do_select()** 原生上下文中调用是安全的。代价是需使用自编译内核，对本文固定的实验平台不构成额外负担。

[来源：实验/04_ 实验环境与测量工具校准/结论.md, 内核数据搜集分析.md]

2.6 实验硬件与软件配置

本实验平台由树莓派 5、SO-101 机械臂和两路 USB 摄像头组成。硬件侧实际搭建了两个 SO-101 follower，每个 follower 含 6 个舵机，并由一块舵机驱动板统一控制；在线策略推理实验只使用其中一个 follower。两路摄像头分别作为手眼相机和固定相机接入，均通过 OpenCV 采集。

项目	配置
推理主机	树莓派 5, ARM Cortex-A76, 4 核, 4 GB LPDDR4X, 无 GPU
机械臂	SO-101 follower ×2; 推理时使用其中 1 个 follower
舵机与驱动	每个 follower 含 6 个舵机，由 1 块舵机驱动板控制
摄像头	USB 摄像头 ×2, 位置分别为 handeye 和 fixed
图像采集	OpenCV Camera, 分辨率 640×360, 30 fps
控制参数	chunk_size =100, 控制目标 fps=30, 串口波特率 1 Mbps
采集设置	每类任务采集 ≥ 3 次 episode 取均值，排除冷启动影响

软件/库	版本或说明
操作系统	Raspberry Pi OS
Python	3.10
LeRobot	0.3.4
PyTorch	2.7.1
torchvision	0.22.1
OpenCV	4.12.0.88
NumPy	2.2.6
pyserial	3.5
feetech-servo-sdk	1.0.0

第三章 性能分析

本章围绕 LeRobot 在线推理循环中的三个主要阶段展开：感知阶段负责采集两路相机图像和机械臂关节状态，推理阶段负责 ACT 策略前向计算，执行阶段负责将动作通过舵机总线下发到 SO-101 follower。三者虽然在 Python 主循环中串行出现，但其底层开销来源并不相同：摄像头路径主要经过 OpenCV/V4L2 和用户态图像解码，推理路径主要受 ARM CPU 上 PyTorch 算子计算限制，舵机路径则受半双工串口总线和 SCServo 协议约束。

3.1 感知阶段性能剖析

感知阶段对应主循环中的 `robot.get_observation()`。在本文的 SO-101 实验配置中，一次 observation 由两类数据组成：两路 OpenCV 摄像头图像，以及一个 follower 机械臂的 6 个舵机当前位置。其中两路摄像头分别为 `handeye` 和 `fixed`，分辨率均为 640×360 ；舵机状态来自 6 个 STS3215 舵机的 `Present_Position` 寄存器。

从代码结构看，SO-101 follower 的观测函数先同步读取 6 个舵机位置，随后遍历两路相机，取回后台线程已经预取的图像。这意味着 obs 阶段并不是单一 I/O，而是“舵机同步读 + 两路相机异步取帧”的组合。舵机读取与执行阶段共用同一条 `/dev/ttyACM0` 半双工总线，协议和内核路径高度重合，因此本节只点明其在 obs 中的位置，详细分析放在 §3.3。

3.1.1 摄像头异步读取机制

LeRobot 的 OpenCV 相机并不是在主线程里每次直接阻塞读取硬件帧。每个相机对象内部维护一个后台读线程：只要没有收到停止信号，它就循环调用 OpenCV 读取摄像头，把最新图像写入共享缓存，并设置帧就绪事件。主线程中的 `async_read()` 不再直接面对摄像头设备，而是等待这个事件；事件到达后，它只需短暂获取锁，从共享缓存取走最近一帧，然后清除事件标志，等待下一次更新。这里不是两个线程互相通知：后台线程负责生产帧并设置事件，主线程负责等待事件并清除事件。

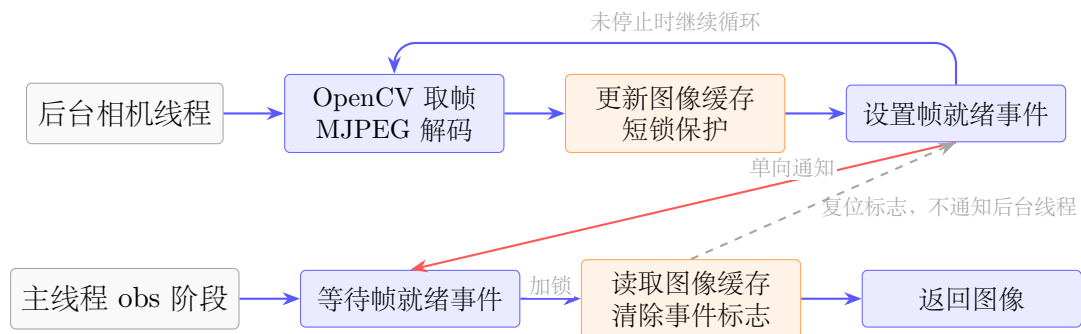


图 7: OpenCV 摄像头异步读取中的后台循环、单向事件通知与主线程取缓存关系

后台线程会调用 OpenCV 的读帧接口，从 V4L2 摄像头设备取出一帧，并在用户态完成 MJPEG 解码和颜色格式整理。整理后的 RGB 图像被写入 `latest_frame`，这个共享缓存由 `frame_lock` 保护。`new_frame_event` 只承担“帧已就绪”的单向通知作用；主线程取走缓存帧后会复位事件标志，避免下一轮直接读到旧帧，但这个复位动作不会反向唤醒或驱动后台相机线程。

因此，`async_read()` 的“异步”含义是：摄像头硬件读取和 MJPEG 解码提前在后台线程中进行，主线程在 obs 阶段只等待事件并读取最近一帧。如果后台线程已经准备好图像，主线程很快返回；如果主线程跑得比相机帧率更快，则会在等待新帧事件时阻塞。

3.1.2 是否进入内核

摄像头路径中有三类等待，需要区分其含义。

位置	是否可能进入内核	含义
主线程等新帧事件	可能进入内核	线程同步等待。对应 <code>new_frame_event.wait()</code> ，主线程等后台相机线程把新帧准备好
主线程取缓存帧	通常不进入内核	短锁保护。对应 <code>frame_lock</code> ，只是在读写 <code>latest_frame</code> 时避免两个线程同时改同一块缓存
OpenCV 后台线程取帧	会进入内核	设备可读等待。后台线程通过 <code>select()</code> 等待 V4L2 摄像头有帧可取，随后再执行出队操作取得该帧

也就是说，主线程的 `async_read()` 主要是在等 Python 线程同步事件；真正深入 V4L2、USB 摄像头驱动和 videobuf2 队列的是后台读线程。这也是为什么主线程观测耗时可能只有毫秒级，而摄像头后台线程的 `pselect6` 等待会接近 30 fps 的帧间隔。

3.1.3 OpenCV 到内核的边界

论文正文不展开完整源码栈，只保留性能分析需要的边界关系：后台相机线程首先进入 OpenCV 的 `VideoCapture`，随后通过 glibc 发起 V4L2 相关系统调用，最终进入 Linux 的 V4L2、UVC 和 videobuf2 队列。

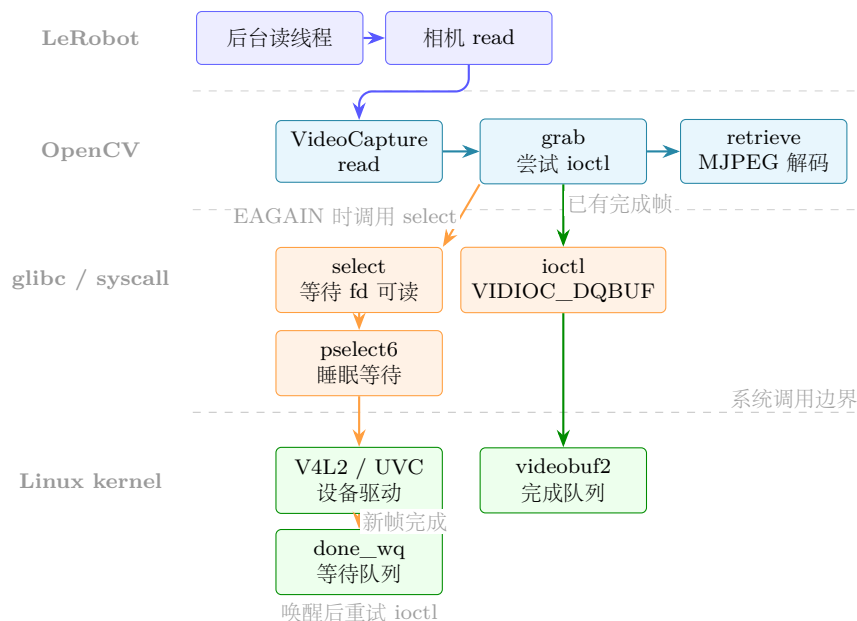


图 8: OpenCV 摄像头后台线程从用户态到 V4L2 内核队列的关键路径

图 8 中有两条需要区分的路径：等待设备可读通常发生在 `select()` / `pselect6` 一类调用上；真正取出已完成图像缓冲区时，OpenCV 使用 `VIDIOC_DQBUF` 将一个已经由驱动填好的 V4L2 buffer 从完成队列中出队。

需要注意的是，`VIDIOC_DQBUF` 并不是把整帧像素从内核重新拷贝一遍。在当前 OpenCV V4L2 路径中，摄像头 buffer 通过 `mmap` 暴露给用户态；出队操作主要返回 buffer 的编号、有效字节数和时间戳等元数据。MJPEG 到 BGR 图像的解码仍然发生在用户态 OpenCV/libjpeg 中，这部分才是 ARM CPU 上摄像头链路中更明显的计算开销。

完整的 OpenCV、glibc 和 Linux 内核源码级 CallStack 与参数说明，不放在论文正文中展开，单独记录在 robotdoc/摄像头相关/OpenCV 到 glibc 到内核 CallStack 源码解析.md。

3.1.4 时间剖析

在 30 fps 摄像头配置下，单路摄像头物理帧间隔约为 33.3 ms。实测中，后台线程在 `pselect6` 或 V4L2 等待路径中阻塞约 23–32 ms，符合摄像头帧率上限；MJPEG 解码仍在用户态 OpenCV/libjpeg 中完成，在 ARM CPU 上属于摄像头链路的主要计算开销。由于 LeRobot 采用后台预取，主线程 obs 阶段看到的是“取缓存帧”的时间，而不是完整的摄像头端到端采集时间。因此，主线程通常不会被摄像头硬件读取、V4L2 等待或 MJPEG 解码直接阻塞；这些耗时主要由后台相机线程承担。主线程只有在请求图像时后台线程尚未准备好新帧，才会短暂等待 `new_frame_event`，随后只是持锁读取最近缓存帧。

实测的逐层耗时分布与分层架构图见实验 06(摄像头感知层级耗时剖析,robotdoc/实验/06_摄像头感知层级耗时剖析/)。该实验通过 `ftrace` 内核插桩和 Python `perf_counter` 双层时间戳，量化了从 `pselect6` 阻塞到 tensor 入模的层级耗时，并区分了“端到端时延”与“主线程感知耗时”在异步预取下的差异。

[来源：OpenCV 视频流内部实现.md，OpenCV 到 glibc 到内核 CallStack 源码解析.md，pselect6 最终结果.md，实验数据/06_摄像头感知层级耗时剖析/README.md]

3.2 推理阶段性能剖析

本节专门分析 SO-101 在线控制循环中的 ACT 策略推理阶段。这里的“推理阶段”并不只是一次 `ACT.forward()`，而是从 `predict_action()` 收到 observation 开始，经过动作缓存判断、必要时的观测预处理、ACT 模型前向、动作队列写入、再到动

作反归一化返回的完整路径。

当前实验配置中，ACT 的 `chunk_size` 与 `n_action_steps` 均为 100，单次完整模型推理输出 $(B, \text{chunk_size}, \text{action_dim}) = (1, 100, 6)$ 的动作序列。因此在线循环的大多数帧并不会重新执行模型前向，而是直接从 `_action_queue` 中取出上一次推理缓存的下一帧动作。这一区别是理解第 3.2 节所有时间数据的前提：`chunk_size` 主要是在时间上分摊单次 ACT 前向开销，而不是让一次 ACT 前向本身变快。

3.2.1 select_action 与动作队列

LeRobot 在 `predict_action()` 外层先检查策略对象的动作队列。如果 `_action_queue` 非空，主循环直接 `popleft()` 取出缓存动作，再通过 `postprocessor` 反归一化为真实舵机目标；这一帧会跳过图像 tensor 转换、`preprocessor` 和 `ACT.forward()`。只有当队列为空时，系统才会走完整推理路径：把 numpy observation 转为 PyTorch Tensor，执行归一化预处理，调用 `policy.select_action()`，继而触发 `predict_action_chunk()` 和 `ACT.forward()`。模型一次产生 100 帧归一化动作，写入队列后立即弹出第 0 帧返回；后续 99 帧由队列快速路径逐帧取用。

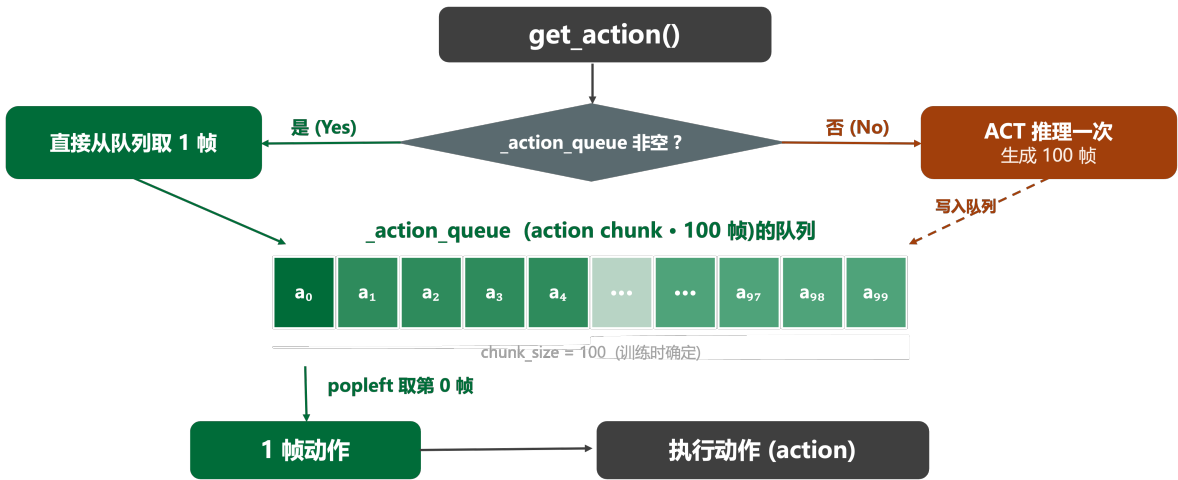


图 9: ACT 在线推理中动作队列快速路径与完整推理路径

图 9 说明了 `chunk_size=100` 下的两种时间尺度：队列非空帧只承担动作取队列和反归一化，属于轻量路径；队列为空帧才承担完整 ACT 前向，属于重推理路径。因此，按 episode 统计得到的 inference 占比，本质上是少数重推理帧在 100 帧执行周期内被均摊后的结果。

3.2.2 一次完整 ACT 推理的数据流

队列为空时，`predict_action()` 会先把原始 observation 整理成 ACT 可接受的 batch。两路相机图像从 (H, W, C) 的 `uint8` numpy 数组转换为 $(1, 3, 360, 640)$ 的 `float32` Tensor；关节状态从 $(6,)$ 转为 $(1, 6)$ 。随后 `preprocessor` 使用随 checkpoint 保存的训练集 mean/std 对图像、state 进行归一化。进入 `ACT.forward()` 后，推理态下不运行训练用 VAE encoder，而是使用全零 latent。

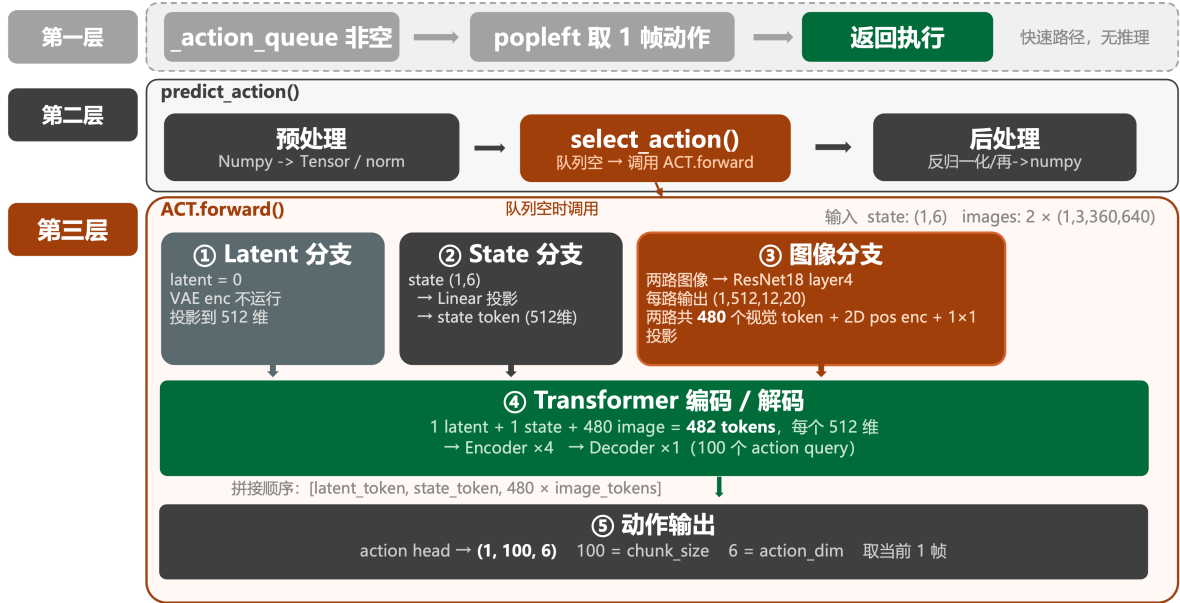


图 10: 队列为空时一次完整 ACT 推理的数据流与模型结构

图 10 中，视觉分支是主要计算来源。每路 640×360 图像经 ResNet18 的 `layer4` 得到 $512 \times 12 \times 20$ 特征图，两路相机共形成 480 个视觉 token；再加上 1 个 state token 和 1 个全零 latent token，构成长度 482、维度 512 的 Transformer Encoder 输入序列。Decoder 使用 100 个可学习 query，最终通过线性层输出 100 帧、每帧 6 维的归一化动作。

3.2.3 时间剖析：chunk_size 参数扫描

实验 01 在树莓派 5 上对同一 ACT 抓取瓶子策略扫描 `chunk_size`，取值覆盖 $\{1, 3, 5, 10, 20, 50, 100\}$ ，每个取值固定记录 20 个 episode，其它参数（模型权重、分辨率、目标 fps）保持一致。原始数据与绘图脚本保存在 [来源：实验 01: `chunk_size` 参数扫描]。后文先看四个阶段的时间占比如何随 `chunk_size` 变化，再单独看 FPS 的增长曲线，最后用代表性数值表汇总。

阶段时间占比随 chunk_size 的演化 图 11 给出了 obs/inference/action/wait 四阶段平均占比（20 episode 均值）随 chunk_size 的变化曲线。inference_pct 从 cs=1 时的 97.6% 单调下降到 cs=100 时的 36.5%，对应 wait_pct 从 0% 单调上升到 43.3%；obs_pct 从近 0% 缓慢增长到 15.4%，action_pct 始终保持在 5% 以内。这条主曲线说明 chunk_size 越大，单帧分摊到的推理工作量就越小，控制循环把更多预算还给 wait（即在帧内等待 fps 节拍），推理不再压满整个 33.3 ms 时间预算。观测和执行阶段的占比上升只是相对结果——其绝对耗时基本恒定，只是因为 inference 缩短才在比例上变得显眼。

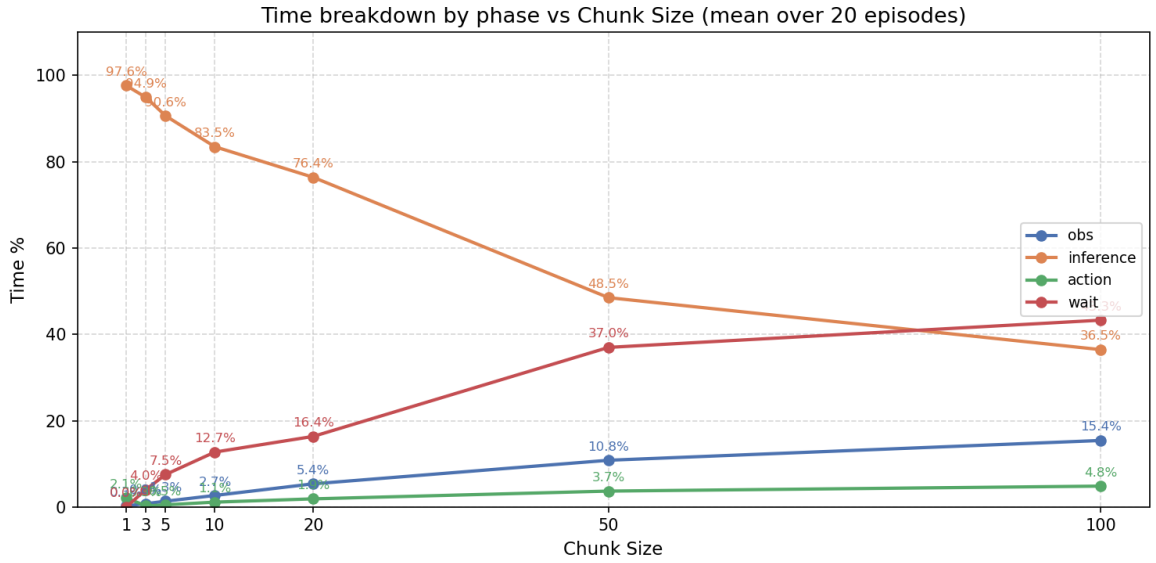


图 11: chunk_size 对控制循环阶段时间占比的影响

FPS 随 chunk_size 的非线性增益 图 12 给出了实测平均 FPS 随 chunk_size 的变化曲线。FPS 从 cs=1 的 0.8 上升到 cs=100 的 18.7，但增长曲线高度非线性：cs=1→10 时 FPS 提升约 6.6 倍（0.8→5.3），cs=10→50 时再提升约 2.9 倍，而 cs=50→100 时增益仅约 1.2 倍（15.4→18.7），呈现典型的边际递减。原因是单次完整 ACT 推理的绝对耗时接近恒定（ $\approx 1-1.5$ s/次），分块机制只是把这一次重推理在时间上摊薄到更多控制帧上，而不是降低 ACT.forward() 本身的计算量；当 cs 增大到推理被摊薄到接近 fps 目标时，等待阶段就成为主循环新的瓶颈，进一步增大 cs 不再带来等比增益。

cs=100 在树莓派 5 上能跑到 ≈ 18.7 fps，距 30 fps 目标仍有约 38% 缺口；而此时 wait_pct 已升至 43.3%，说明继续增大 cs 已经不能进一步把 fps 推到 30——要逼近目标必须从减小单次推理绝对耗时入手，也就是后文（第四章）讨论的优化方

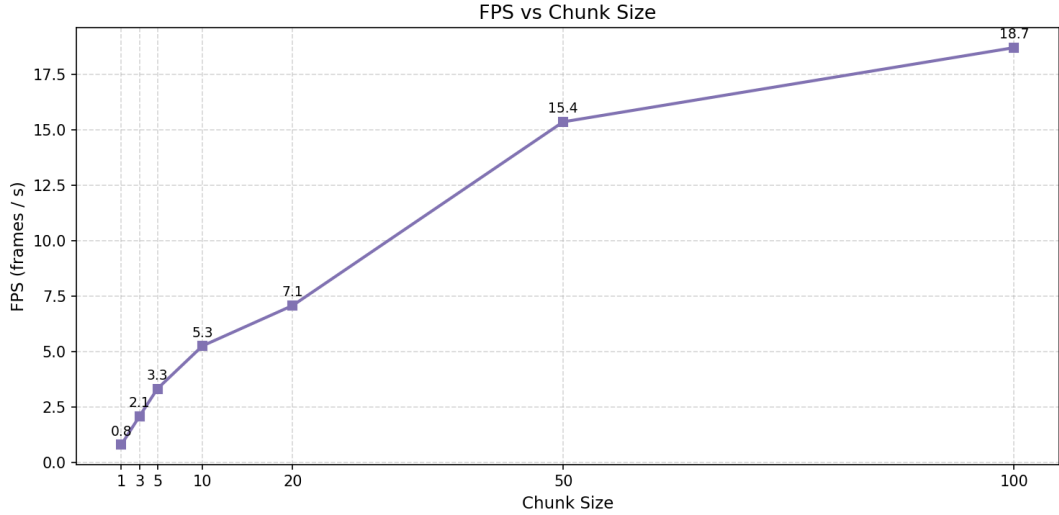


图 12: chunk_size 对控制循环 FPS 的影响

向。

3.2.4 单次 ACT 推理耗时分解

上面的 episode 级统计能够说明 chunk_size 如何分摊推理开销，但还不能回答“一次完整 ACT 推理内部究竟慢在哪里”。实验在 modeling_act.py 的 select_action() 和 ACT.forward() 内插入计时点，通过环境变量 LEROBOT_PROFILING=1 启用，数据直接写入 CSV。在树莓派 5 上记录了 6 次队列为空的完整推理，结果汇总于表 2。

模块	平均耗时 ms	标准差 ms	占完整推理比例
select_action 完整路径	1924.8	414.9	100.0%
ResNet18 + 投影	1281.1	259.9	66.6%
Transformer Encoder	569.5	506.3	29.6%
Transformer Decoder	44.0	4.4	2.3%
action_head	0.7	0.8	<0.1%
队列非空快速路径（对照）	<1 ms	—	—

表 2: ACT 单次完整推理（队列为空时）的逐模块耗时分解

图 13 给出了各模块的耗时柱状图。

关键发现：ResNet18 视觉特征提取（两路 ResNet18 backbone + 位置编码 + 1×1 投影）占据完整推理时间的 **66.6%**（平均 1281 ms），是第一瓶颈。Transformer Encoder 次之，占 29.6%（569 ms）。Decoder 和 action_head 占比极低（2.3% 和 <0.1%），不是优化重点。队列非空快速路径耗时不足 1 ms，与后处理相当，验证了动作缓存机制的有效性。

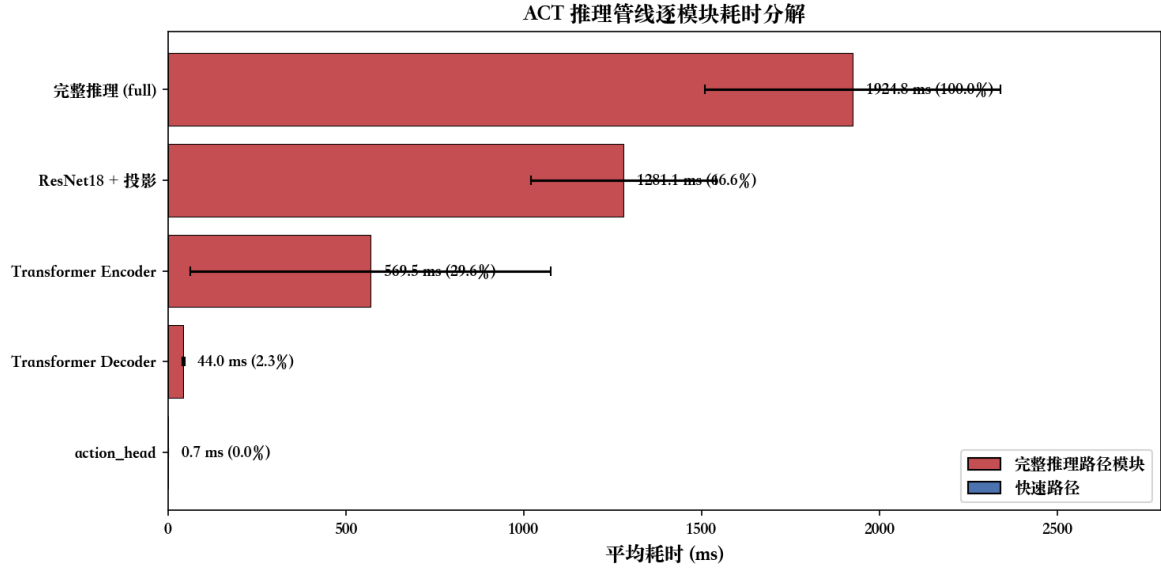


图 13: ACT 推理管线逐模块耗时分解

3.2.5 PyTorch 线程行为与推理优化研究现状

PyTorch 在 ARM CPU 上使用 OpenMP 并行框架，通过 ATen 线程池调度矩阵运算。系统调用追踪显示 futex 调用共计 64,569 次 / 40 s episode，其中 27 线程并行累计等待时间达 3,252 s，说明多线程同步开销在 ARM CPU 上不可忽略。

推理优化方向（如 INT8 量化、算子融合、线程亲和性、NEON SIMD 加速等）已有大量文献研究，此处不再展开。

3.3 执行阶段性能剖析

执行阶段的核心是机械臂六个 STS3215 舵机的位置指令下发（`sync_write`）。为完整描述舵机链路，本节同时纳入感知阶段中物理通道完全重合的舵机位置读取（`sync_read`）。两者共用同一条 `/dev/ttyACM0` 半双工 TTY 总线、相同的 SCServo 协议帧结构与 USB 串口内核路径，仅在数据包内的指令字节（0x82 vs 0x83）和方向（写 vs 读 + 回包）上不同。

3.3.1 舵机感知读取（`sync_read` 路径）

调用链：`motors_bus.sync_read()` → `GroupSyncRead` → `PortHandler` → `/dev/ttyACM0`。

- Sync Read 包格式：[0xFF 0xFF 0xFE LEN 0x82 ADDR LEN ID1..ID6 CHK]；
- Present_Position 寄存器地址 56，sign-magnitude 编码（bit15 为方向位）；
- 耗时：1.2 ms/帧，开销可忽略。

完整调用链：

- 用户态入口: `motors_bus.sync_read()` (`scservo_sdk`)
- SDK 层: `GroupSyncRead::txPacket()` → `PortHandler::writePort()`
- 系统调用: `write(fd, buf, len)` → `sys_write()` → TTY 层
- 内核路径: `tty_write()` → `usb_bulk_msg()` → `/dev/ttyACM0`
- 回包路径: `read()` → `tty_read()` → `usb_bulk_msg()` → 用户态
- 协议解析: `GroupSyncRead::rxPacket()` 解析返回位置数据

[来源: *observation* 读取 *ACM0* 数据完整流程.md, *Lerobot* 内舵机读写完整链路.md, *USB* 串口写路径.md]

3.3.2 舵机指令下发 (sync_write 路径)

调用链: `motors_bus.sync_write()` → `GroupSyncWrite.txPacket()` → `write()`
→ TTY → USB bulk transfer。

- 无回包，半双工 TTY 总线，耗时 **1–3 ms**；
- 开销可忽略，不是系统瓶颈。

完整调用链：

- 用户态入口: `motors_bus.sync_write(position_dict)` (`scservo_sdk`)
- SDK 层: `GroupSyncWrite::txPacket()` 组包 → `PortHandler::writePort()`
- 协议层: 半双工 TTY 总线，`write()` 调用后等待 1–3 ms 完成
- 系统调用: `write(fd, buf, len)` → `sys_write()` → `tty_write()`
- 内核层: `usb_bulk_msg()` → `dwc_otg` USB 驱动 → 物理 USB 传输
- 半双工约束: 总线冲突保护，同一时刻仅允许一个方向传输

[来源: *USB* 串口写路径.md, *motors_bus* 异步读写设计分析.md, *Lerobot* 内舵机读写完整链路.md]

3.3.3 半双工总线协议特性与瓶颈

关键性能结论（融合 `sync_read` 与 `sync_write` 两条路径的共性）：

- 系统调用本身开销可忽略：`write` 系统调用耗时仅 $38\ \mu\text{s}$ ，`read` 类似量级，二者组合（`sync_read`）约 $1.2\ \text{ms}$ ，单 `write`（`sync_write`）约 $1\text{--}3\ \text{ms}$ ；
- 瓶颈在物理层而非软件路径：1 Mbps 波特率下，Sync Read 帧 14–15 字节、Sync Write 帧约 28 字节（ $6\ \text{ID} \times 4\ \text{字节位置} + \text{包头} + \text{校验}$ ），总线传输时间是耗时主体；
- 半双工约束是异步化的根本障碍：同一时刻总线仅能容纳一个方向的传输，异步并发会导致总线冲突、指令帧损坏，且 `send_action()` 中 read-modify-write 的安全限幅链路依赖同步读返回的实时位置，异步化将破坏这一不变量（详见 §4.3）。

第四章 优化

4.1 性能瓶颈归因

- 主瓶颈：ACT 推理（ARM CPU 矩阵乘法）， $2000\text{--}4000\ \text{ms/次}$ ；
- 次瓶颈：MJPEG 解码（ $10\text{--}30\ \text{ms/帧}$ ，可替换为 YUYV 消除）；
- 非瓶颈确认：系统调用本身（`pselect6` $421\ \text{ns}$ ，`futex` $<1\ \mu\text{s}$ ，均远低于推理耗时）。

4.2 跨阶段综合分析

4.2.1 时间预算表（单帧， $33.3\ \text{ms}$ 目标）

阶段	实测耗时	占目标比例
感知（舵机 + 图像，主线程视角）	$5.2\ \text{ms}$	15.6%
推理（chunk 内均摊）	[TODO:] ms	[TODO:] %
执行（ <code>sync_write</code> ）	$4.8\ \text{ms}$	14.4%
其余调度开销	[TODO:] ms	[TODO:] %
实际单帧循环（推理帧）	$2200\text{--}4200\ \text{ms}$	$66\times\text{--}126\times$ 超时

4.2.2 系统调用分布（40 s episode）

系统调用	次数	均值耗时	主要来源
futex	64,569	$<1\ \mu\text{s}$	PyTorch 线程池
pselect6	36,191	421 ns	摄像头后台线程 V4L2 等待
write	35,651	$38\ \mu\text{s}$	USB 串口写

结论：系统调用本身开销极低，瓶颈在用户态 CPU 推理计算，非 IO。[来源：*recordloop* 每部分时间全部分析.md，内核数据搜集分析.md]

4.3 异步化可行性调研

4.3.1 摄像头异步读

LeRobot 摄像头采集通过 OpenCV 后台线程实现异步：主线程调用 `retrieveFrame()` 时，后台线程已在 `pselect6()` 等待下一帧。

- **当前状态：**已是默认实现，无需改造；
- **可优化点：**MJPEG 解码（10–30 ms/帧）为 CPU 密集瓶颈，切换为 YUYV 格式可直接消除解码开销；
- **结论：**摄像头异步读已实现，当前主要矛盾在解码格式而非异步化。

4.3.2 推理异步

LeRobot 支持双进程推理架构（主进程 + 推理进程，通过 gRPC 通信）：

- **当前状态：**已实现但未启用（当前配置下 stall 不可避免）；
- **不 stall 条件推导：**

$$N \geq 2 \times T_{\text{infer}} \times F$$

代入参数（ $T_{\text{infer}} = 2\text{ s}$ ， $F = 30\text{ fps}$ ， $N = 100$ ）：

需要 $N \geq 120$ ，当前 $N = 100 < 120 \Rightarrow$ stall 不可避免

- **结论：**配合 INT8 量化（预期 T_{infer} 降至 0.5–1 s）后，可将不 stall 所需 N 降至 30–60，当前 $N = 100$ 可满足条件，推理异步方可启用。

[来源：*async_inference_* 推理执行流水线分析.md]

4.3.3 舵机异步读

舵机位置读取通过 `sync_read` 实现（半双工写请求 + 读回包），异步化改造存在以下难点：

- **总线半双工约束：**读写操作必须串行，异步并发写入会导致总线冲突；
- **安全限幅链路：**`send_action()` 需先读取当前位置（read-modify-write），若读操作异步化，则无法保证限幅判断的时效性；
- **结论：**在当前半双工总线协议下，舵机异步读难以实现，不建议作为优化方向。

[来源：Lerobot 内舵机读写完整链路.md, *motors_bus* 异步读写设计分析.md]

4.3.4 舵机异步写

类似异步读，舵机指令下发（`sync_write`）的异步化存在根本性约束：

- **总线半双工冲突：**异步并发写会导致总线冲突，指令帧损坏；
- **安全限幅链路断裂：**`send_action()` 中的 read-modify-write 操作依赖同步读返回的实时位置，若写操作先于读完成，限幅判断将基于过期数据，可能导致机械臂碰撞；
- **相关工作：**调研现有机器人控制系统异步执行架构`async_robot`，均在支持全双工或具备独立控制通道的硬件平台上实现，不适用于半双工 TTY 总线；
- **结论：**舵机异步写在当前硬件平台上不可行，亦非必要（写操作本身仅 1–3 ms）。

[来源：*motors_bus* 异步读写设计分析.md]

4.4 优化方向矩阵

优化方向	预期收益	可行性	复杂度
INT8/FP16 量化	推理降 2–4×	高	中
ONNX Runtime 后端	推理降 1.5–2×	高	低
MJPEG → YUYV	解码降至 0	高	低
推理异步（gRPC 双进程）	消除 stall(需先完成量化)	中	高
异步舵机读	不可行	×	—
异步舵机写	不可行	×	—

4.5 软件架构改进方向

- 推理模块解耦（独立进程 + gRPC），避免 GIL 与主循环相互干扰；
- 摄像头格式统一为 YUYV，消除 libjpeg CPU 解码路径；
- 面向机器人负载的内核优化方向：实时调度器（SCHED_FIFO）、USB 驱动优先级提升。

第五章 总结与展望

5.1 主要工作总结

1. 建立了覆盖 AI 框架到内核的四层性能分析模型；
2. 通过 Python 日志 + ftrace + torch.profiler 多工具联合，端到端量化了各阶段开销；
3. 揭示 ACT 推理（2000–4000 ms）为主瓶颈，系统调用 overhead 可忽略；
4. 推导了异步推理流水线不 stall 的数学条件，指出当前参数配置的局限性；
5. 系统梳理了各优化路径的可行性与成本收益；
6. 对感知、推理、执行三阶段进行了从 Python 到 kernel 的全链路 CallStack 源码分析；

7. 调研了各阶段异步化可行性，论证了舵机异步读/写的不可行性及其原因。

5.2 研究局限性

- 仅在 SO101 单臂场景下测试，未覆盖双臂或移动机器人；
- 未实施量化等优化方案并验证，结论停留在分析层面；
- 无 GPU 对比基线，难以量化 CPU 推理与加速器的差距。

5.3 未来工作

- INT8 量化 + ONNX Runtime 实验，验证推理加速收益；
- 扩展至多任务场景（更多抓取类别）与多机器人平台；
- 探索面向机器人负载特征的实时操作系统设计。

参考文献

- [1] BROHAN A, BROWN N, CARBAJAL J, et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control[J]. arXiv preprint arXiv:2307.15818, 2023.
- [2] KIM M J, PUMACAY K, JATAVALLABHULA K M, et al. OpenVLA: An Open-Source Vision-Language-Action Model[J]. arXiv preprint arXiv:2406.09246, 2024.
- [3] CADENE R, ALIBERT S, et al. LeRobot[EB/OL]. 2024. <https://github.com/huggingface/lerobot>.
- [4] ZHAO T Z, KUMAR V, LEVINE S, et al. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware[C]//Proceedings of Robotics: Science and Systems (RSS). 2023.
- [5] CHI C, FENG S, DU Y, et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion[C]//Proceedings of Robotics: Science and Systems (RSS). 2023.
- [6] INTELLIGENCE P.
pi0: A Generalist Robot Policy[EB/OL]. 2024. <https://www.physicalintelligence.com/>.
- [7] ZHAN T, et al. Mobile ALOHA: Bimanual Mobile Manipulation with Low-Cost Whole-Body Teleoperation[C]//IEEE International Conference on Robotics and Automation (ICRA). 2024.
- [8] ROBOTIS. OpenManipulator[EB/OL]. 2024. https://emanual.robotis.com/docs/en/platform/openmanipulator_x/.
- [9] ROBOTICS O, ROBOTICS C. TurtleBot 4[EB/OL]. 2023. <https://www.turtlebot.com/>.
- [10] HybridRobotics. Berkeley Humanoid Lite[EB/OL]. 2025. <https://lite.berkeley-humanoid.org/>.

附 录

附录 A：待补充实验清单

优先级	实验内容	填充章节	预计耗时
★★★	推倒/分类任务计时数据	第 3.2.2 节三类任务对比	已完成
★★★	torch.profiler op-level	第 3.2.3 节算子分解	1 天
★★★	摄像头层级耗时剖析（实验 06）	第 3.1.2 节 Call-Stack 源码分析	1 天
★★	CPU 频率锁定对比	第 4.2 节峰值成因	半天
★	perf stat (IPC, cache-miss)	判断 compute vs memory bound	半天

致 谢

[TODO: 致谢内容]